Preface

If you write Rust, you already spend a good part of your day in conversation with a type checker. It tells you that a value was moved, that a reference does not live long enough, that a trait is not implemented for a type, that two branches of an `if` disagree. Over time you learn to anticipate it, and then to use it: you design types so that the checker rejects the mistakes you are afraid of making. At that point a natural curiosity sets in. *Why* are these the rules? Where do they come from? Is there a theory behind them, and could you learn it?

There is, and you can. This book teaches it.

The theory is called **type theory**, and it has two close relatives that turn out to be the same subject seen from different angles: **logic**, where types are propositions and programs are proofs, and **category theory**, where types are objects and programs are arrows between them. Behind Rust's most distinctive feature, ownership, stands a fourth subject that grows out of the first three: **substructural logic**, the logic of resources that can be used at most once.

Most books on these subjects use Haskell, OCaml, Coq or pure mathematics. This one uses the language you already know, and a second language that makes the theory concrete.

Two languages

Rust is where the ideas live in production. Its type system is unusually rich: algebraic data types, generics that behave like System F, traits that are type classes, `impl Trait` and `dyn Trait` that are existential types, an affine ownership discipline, lifetimes that are regions, variance, const generics. Each of these is a chapter of type theory, and Rust lets you poke at it: write a program, see whether it compiles, read the error. Rust also deviates from the clean theory in instructive ways (every type is inhabited by `panic!()`, type inference stops at function boundaries, there are no higher-kinded types, the borrow checker is a dataflow analysis rather than a type system), and the book pays as much attention to the deviations as to the matches.

Lean 4 is where the ideas become precise. Lean is a functional programming language with *dependent types*, types that can mention values, and that makes it a proof assistant too: a proposition is a type, a proof is a program of that type, and Lean checks the proof the same way it checks any program. In this book Lean is the laboratory. Each toy language we study is defined in Lean, its typing rules are written as Lean definitions, and its properties are stated as theorems and proved. When the book says "the type checker is sound" or "a well-typed program never gets stuck" or "these two types are isomorphic", there is a Lean proof of it in the repository, and Lean has checked every step.

Lean comes with **Mathlib**, the large library of mathematics that its community maintains, which also supplies many of the *tactics* (automated proof steps) that make proofs short and readable. The book uses Mathlib from Chapter 2 on, and introduces it gently. Where building a definition yourself is the best way to understand it (a toy language, a typing judgment, the notion of a category), we build it from scratch first, and then look at the corresponding definition in Mathlib, so that you can carry on with the real library when you finish the book.

The two languages complement each other. Rust implements; Lean proves. A Rust test can check a property on ten thousand inputs; a Lean theorem establishes it for all of them. A Rust type checker is fast and gives good error messages; a Lean formalization of the same rules tells you exactly what those rules guarantee.

Who this book is for

The book is written for programmers who:

- use Rust actively: ownership and borrowing, structs and enums, traits and generics, closures, iterators, `Result`-based error handling and Cargo. You do not need to be an expert. When the book uses one of Rust's less familiar features (variance, higher-ranked trait bounds, generic associated types, `PhantomData`, `unsafe`), it explains it.
- know little or nothing about Lean, type theory, logic or category theory. Everything is built from first principles: the first inference rule appears in Chapter 1, the first Lean program in Chapter 1, the first category in Chapter 16.
- want to *understand*: to see the real definitions and the real proofs, not a tour of slogans. The book does not water the subject down. It goes slowly instead, and it explains every step.

You will need some patience with notation and a willingness to read a proof line by line. You will not need any mathematics beyond what a programmer already uses: induction on natural numbers and on trees, sets, functions and the idea of a proof. Everything else is introduced when it is needed.

What you will build

Every chapter is organized around code that compiles, runs, or is checked by Lean. Among other things you will build:

- a sequence of **toy languages**, each implemented in Rust and formalized in Lean: a small expression language with a type checker proved sound; the untyped λ-calculus with Church numerals; the simply typed λ-calculus, with a bidirectional checker in Rust, an intrinsically typed evaluator in Lean and a complete mechanized proof of type safety; a mini-ML with Hindley–Milner type inference;
- **proof tools**: a natural-deduction proof checker, a search for Kripke countermodels that shows which classical laws fail constructively, and a proof search for linear logic that shows why $A \vdash A \otimes A$ is not provable;

- a **library of category theory** in Lean: categories, functors, natural transformations, the Yoneda lemma, cartesian closed categories, the Kleisli category of a monad, each with its laws proved;
- the **formal core of ownership**: an affine λ-calculus with `Copy` types and its "use at most once" theorem, a model of drops, typestate protocols, a core calculus of borrows and lifetimes with derivations checked in Lean, and a miniature MIR with a non-lexical borrow checker that reproduces rustc's errors;
- **verified programs**: a sort and a binary search proved correct in Lean, with Rust implementations tested against the verified models, and a small separation logic.

How the book is organized

Part I — Foundations (Chapters 1–6) introduces the two languages, then judgments, inference rules and derivations, natural deduction, intuitionistic logic and the meaning of constructive proof, and the structural rules.

Part II — Programs as Proofs (Chapters 7–11) covers the λ-calculus, the simply typed λ-calculus and Rust's closures, a full proof of type safety, the Curry–Howard correspondence, and algebraic data types.

Part III — Abstraction (Chapters 12–15) covers parametric polymorphism, type inference, traits and type classes, and existential types.

Part IV — Categories (Chapters 16–20) introduces category theory for programmers: categories, functors and higher-kinded types, natural transformations and free theorems, cartesian closed categories, and monads.

Part V — Ownership as Logic (Chapters 21–25) covers substructural logics, the affine λ-calculus, `Copy` and `Clone` as the exponential modality, `Drop` as weakening with a side effect, and typestate.

Part VI — Borrowing and Regions (Chapters 26–29) covers references and regions, subtyping and variance, a core calculus for borrowing, and how rustc really checks borrows.

Part VII — Beyond (Chapters 30–33) covers dependent types, proving programs correct, soundness and semantic types, and ends with the verdict on the question asked in Chapter 1.

The **appendices** cover setup, a Lean cheat sheet for Rust programmers, notation and a dictionary between the fields, the tactics used in the book, a glossary and further reading.

Read Part I first, and Chapter 2 in particular: it is your introduction to Lean. Part II should come next. After that the parts can be read in several orders: Part IV needs only Parts I and II; Parts V and VI need Parts I and II and are best read together; Part VII draws on everything. Chapter 1 has a map.

Every chapter starts with a short list of what you will learn. It then explains the concepts, walks through complete examples, discusses common confusions, and ends with a summary and exercises with hints. The exercises mix Rust, Lean and pen-and-paper work; doing a few of them in each chapter is the fastest way to make the material your own.

Conventions

Code that you can run or check is always part of the book's repository. When a listing comes from a file, the text names the file the first time, for example `code/rust/examples/ch01_first_types.rs` or `code/lean/Book/Ch01/FirstSteps.lean`. Listings are copied from these files when the book is built, so the code you read is the code that compiles.

Commands you type in a terminal look like this:

```
cargo run --example ch01_first_types
```

The output of a Rust program is shown in a plain block. Every such block is a snapshot of a real run, and the repository's tools check that the program still produces exactly that output:

```
add(2, 3) = 5
```

Some Rust programs in the book are *supposed* to be rejected, because the rejection is the point. They live in `code/rust/compile_fail/`, and the diagnostics shown in the book were captured from the real compiler:

```
error[E0308]: `if` and `else` have incompatible types
```

Lean code is shown with its output written right above the command that produces it, in a special comment that Lean itself checks. In this block, Lean evaluates `2 + 3`, and the build fails unless the result is exactly the `5` written in the comment:

```
/-- info: 5 -/
#guard_msgs in
#eval 2 + 3
```

The same mechanism is used to show the errors Lean reports for programs and proofs it rejects. Chapter 1 explains it in detail.

Mathematics is written in the usual notation. Inference rules are written as fractions with the premises above the line, the conclusion below it and the name of the rule on the right:

$$\frac{\Gamma \vdash f : \sigma \to \tau \qquad \Gamma \vdash a : \sigma}{\Gamma \vdash f\ a : \tau}\ \text{T-App}$$

Appendix C lists every symbol used in the book.

Throughout the text you will find a few kinds of highlighted notes:

> **NOTE**
>
> Background information, history, or a detail that you can skip on a first reading.

> **TIP**
>
> A practical shortcut or a good habit.

> **WARNING**
>
> A common mistake or a subtle point that is easy to get wrong.

The code and how to run it

The repository has this layout:

```
book/              the chapters (Markdown sources, generated Markdown and PDFs)
code/
├── rust/          a Cargo workspace: examples/, crates/, compile_fail/
└── lean/          a Lake project: Book/Ch01/ ... Book/Ch33/
tools/             scripts that build the book and verify the code
```

Both toolchains are pinned: `code/rust/rust-toolchain.toml` selects Rust 1.99.0 and `code/lean/lean-toolchain` selects Lean 4.34.1. If you use rustup and elan (Appendix A explains how to install them), the right versions are downloaded automatically the first time you run `cargo` or `lake` in those directories.

```
cd code/rust
cargo run --example ch01_first_types          # single-file examples
cargo run -p stlc --example ch08_stlc         # examples that use a crate
cargo test -p arith                           # a crate's tests

cd ../lean
lake exe cache get                            # once: download prebuilt Mathlib
lake build                                    # check every Lean file
```

The Rust code uses only the standard library, so it builds in seconds. The Lean code depends on Mathlib, which is big: `lake exe cache get` downloads the prebuilt library once (under a gigabyte to download, about 8 GB on disk with its sources), after which checking the book's Lean files takes a few minutes. To verify the whole book at once (every example, every rejected program, every Lean file):

```
python3 tools/run_examples.py
```

The best way to read the Lean parts of the book is with the Lean files open in an editor (VS Code with the Lean 4 extension), so that you can put the cursor inside a proof and watch the goals change. Appendix A explains how to set this up.

A note on versions and accuracy

Rust is stable and backwards compatible; the programs in this book should keep compiling for a long time, although the exact wording of compiler diagnostics changes between releases. Lean 4 is younger and changes faster: names in its standard library are occasionally renamed, tactics grow more powerful, and the format of error messages evolves. Everything in this book was checked with the pinned versions. If you use a newer Lean and a proof breaks, the `lean-toolchain` file lets you go back to the version the book was written with.

On the theory side, the book states typing rules "for Rust" in many places. Rust has no official formal semantics, so such rules are *models* of what the compiler does, in the spirit of the research calculi that

formalize Rust's core. The text says so when it matters, and uses the symbol ≈ for "corresponds to, with caveats stated in the text".

Acknowledgements

This book stands on the work of the people who built the subject and the tools: the logicians and computer scientists whose ideas fill its chapters, the Rust project and its contributors, and the Lean community, whose documentation, examples and library made it possible to check every proof in these pages.